126Final_Proj

Taylor MacKenzie

2025-06-08

Part 1: Data Description and Descriptive Statistics

```
library(readr)
```

```
##
## Attaching package: 'readr'

## The following object is masked from 'package:scales':
##
##     col_factor
```

```
diamonds1 <- read.csv("Diamonds Prices2022.csv")
```

1) Select a Random Sample

```
set.seed(06082025)
diamonds_sample <- diamonds1[sample(nrow(diamonds1), 1000), ] # ensures 1000 observations
```

This sample contains exactly 1000 observations, which was the required amount for the project.

2) Describe all the variables

```
library(ggplot2)
summary(diamonds_sample)
```

```
##        X             carat            cut               color
##  Min.   :   51   Min.   :0.2300   Length:1000        Length:1000
##  1st Qu.:13385   1st Qu.:0.4000   Class :character   Class :character
##  Median :27756   Median :0.7000   Mode  :character   Mode  :character
##  Mean   :27326   Mean   :0.8002
##  3rd Qu.:40559   3rd Qu.:1.0400
##  Max.   :53866   Max.   :4.0100
##    clarity              depth           table           price
##  Length:1000        Min.   :53.20   Min.   :44.00   Min.   :  361
```

```
##  Class :character   1st Qu.:61.08   1st Qu.:56.00   1st Qu.:  972  
##  Mode  :character   Median :61.80   Median :57.00   Median : 2312  
##                     Mean   :61.74   Mean   :57.51   Mean   : 3883  
##                     3rd Qu.:62.52   3rd Qu.:59.00   3rd Qu.: 5191  
##                     Max.   :79.00   Max.   :73.00   Max.   :18745  
##        x                y               z        
##  Min.   : 3.840   Min.   :3.870   Min.   :2.400  
##  1st Qu.: 4.710   1st Qu.:4.730   1st Qu.:2.920  
##  Median : 5.640   Median :5.650   Median :3.495  
##  Mean   : 5.727   Mean   :5.731   Mean   :3.536  
##  3rd Qu.: 6.530   3rd Qu.:6.540   3rd Qu.:4.030  
##  Max.   :10.020   Max.   :9.940   Max.   :6.240
```

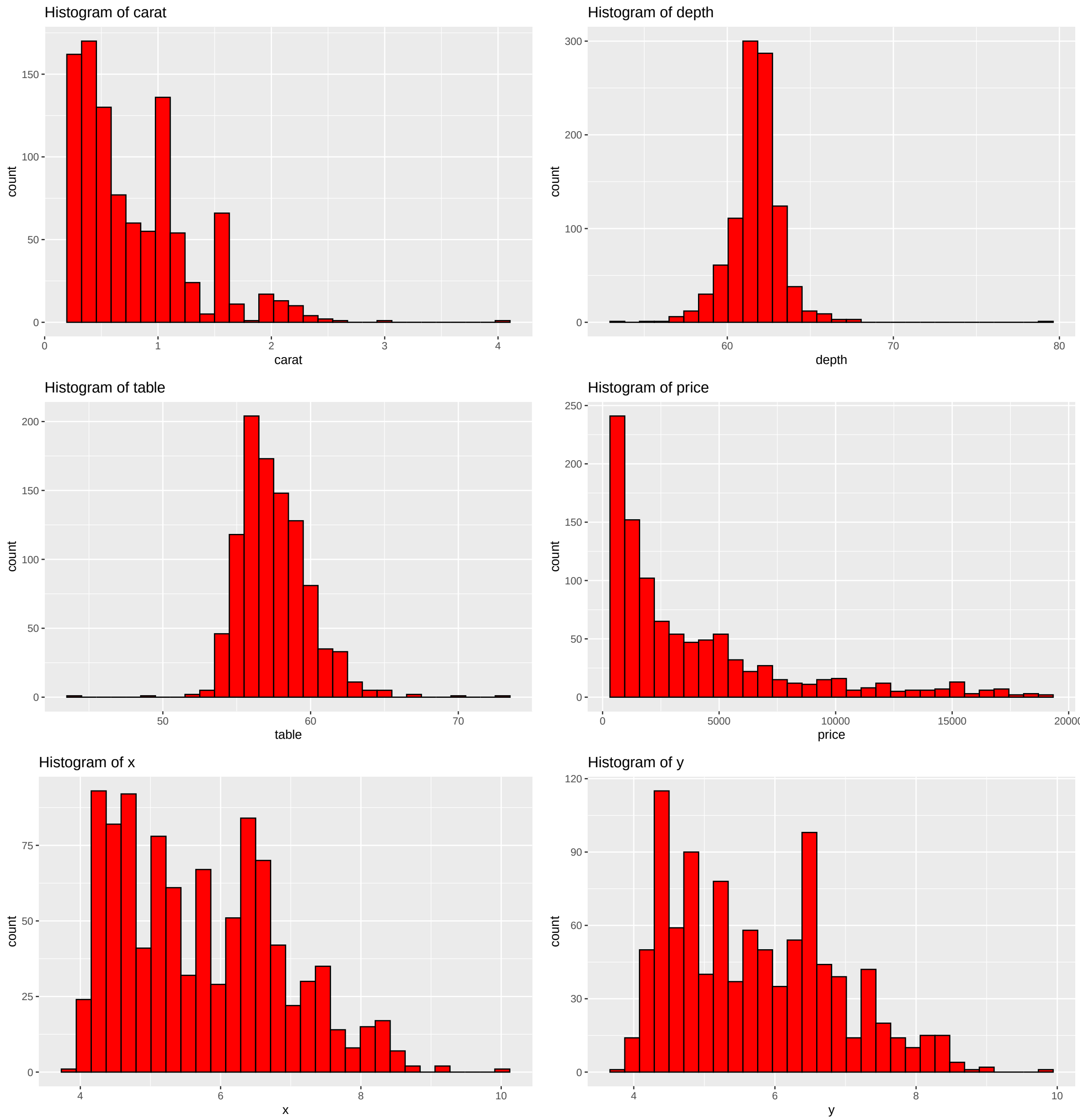
```r
str(diamonds_sample)
```

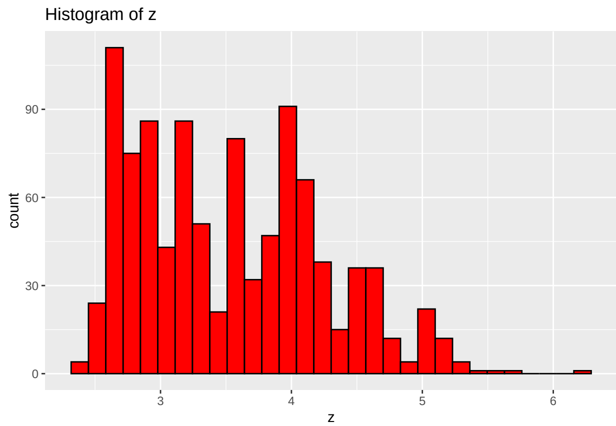
```
## 'data.frame':    1000 obs. of  11 variables:
##  $ X      : int  23080 13365 30133 23838 51471 16122 1276 27585 6805 43413 ...
##  $ carat  : num  1.31 0.24 0.32 1.7 0.84 1.36 0.99 2 1.05 0.53 ...
##  $ cut    : chr  "Very Good" "Very Good" "Very Good" "Good" ...
##  $ color  : chr  "G" "F" "G" "I" ...
##  $ clarity: chr  "VVS2" "VS1" "VS2" "VS1" ...
##  $ depth  : num  62.2 59.9 63.3 58 60.7 61.2 58 61.8 60 61.4 ...
##  $ table  : num  59 58 54 60 56 57 67 56 60 52 ...
##  $ price  : int  11108 419 720 11921 2376 6443 2949 18426 4116 1412 ...
##  $ x      : num  6.91 4.02 4.39 7.84 6.17 7.15 6.57 8.12 6.46 5.29 ...
##  $ y      : num  6.98 4.06 4.36 7.88 6.12 7.19 6.5 8.05 6.43 5.26 ...
##  $ z      : num  4.32 2.42 2.77 4.56 3.73 4.39 3.79 5 4 3.24 ...
```

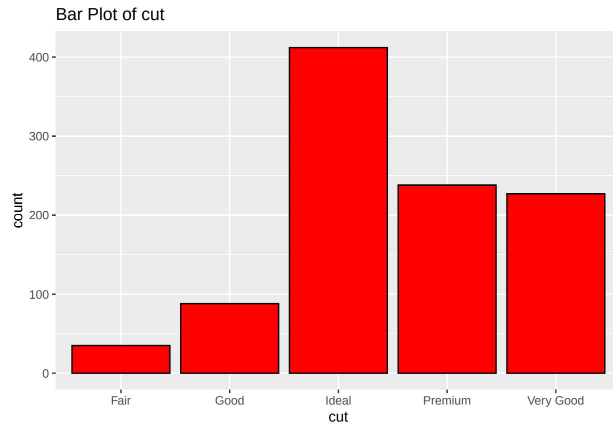
```r
continuous_vars <- c("carat", "depth", "table", "price", "x", "y", "z")
for(var in continuous_vars){
  print(
    ggplot(diamonds_sample, aes_string(x=var)) +
      geom_histogram(bins = 30, fill = "red", color = "black") +
      ggtitle(paste("Histogram of", var))
  )
}
```

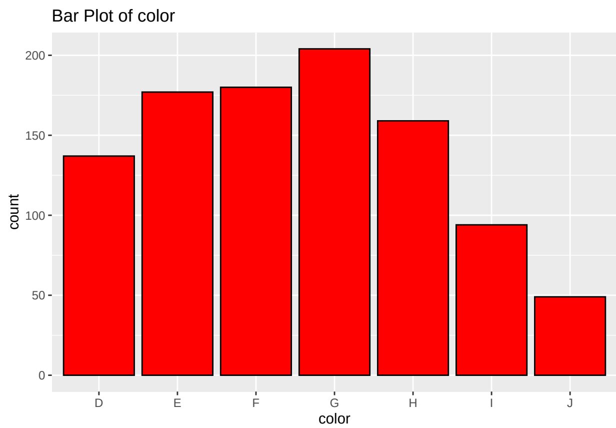
```
## Warning: `aes_string()` was deprecated in ggplot2 3.0.0.
## i Please use tidy evaluation idioms with `aes()`.
## i See also `vignette("ggplot2-in-packages")` for more information.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```

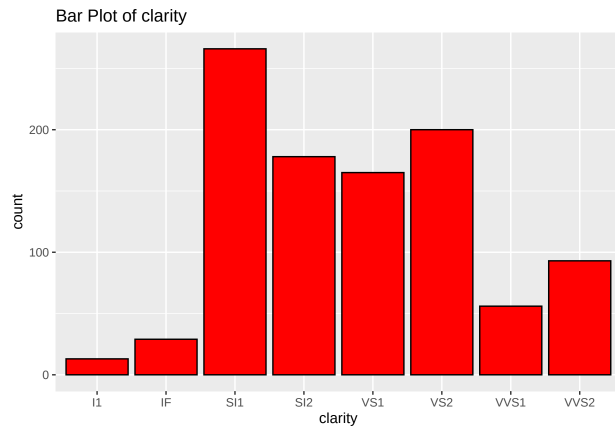
```r
categorical_vars <- c("cut", "color", "clarity")
for(var in categorical_vars){
  print(
    ggplot(diamonds_sample, aes_string(x = var)) +
      geom_bar(fill = "red", color = "black") +
      ggtitle(paste("Bar Plot of", var))
  )
}
```


Histogram of carat
count
0
50
100
150
0
1
2
3
4
carat
Histogram of depth
count
0
100
200
300
60
70
80
depth
Histogram of table
count
0
50
100
150
200
50
60
70
table
Histogram of price
count
0
50
100
150
200
250
0
5000
10000
15000
20000
price
Histogram of x
count
0
25
50
75
4
6
8
10
x
Histogram of y
count
0
30
60
90
120
4
6
8
10
y









The numerical variables in this sample of the dataset are: carat, depth, table, price, x, y, and z. The categorical variables are: cut, color, and clarity. For a more in-depth description of the variables, we start with carat. This is a numeric variable that represents the weight of the diamond in carats, ranging from 0.23 to 4.01 units. The mean is 0.8 with a median of 0.7, the distribution of this variable is right-skewed due to the mean being greater than the median, visualized with the histogram of carat. Next, we have depth, a numeric type variable that represents the total depth percentage (= z/mean(x,y)) with a range of 53.2% to 79% along with a mean of 61.74% and a median of 61.80%. The distribution of depth is very slightly left-skewed. Table is a numeric type variable that represents the width of the top of the diamond relative to its widest point with a range of 44 to 73 units and a mean of 57.51 and a median of 57.00. The distribution of table is slightly right-skewed bell curve, the closest to the normal distribution. Price is a numeric type variable that represents the price in US dollars with a range of $361 to $18,745 along with a mean of $3,883 and a median of $2,312. The distribution of price is right-skewed. The numeric variables x and y represent the dimensions of the diamond (length and width in mm) with an x range of 3.84 to 10.02 and a y range of 3.87 to 9.94. The x mean is 5.727 and the y mean is 5.731. This distribution is slightly symmetric, favoring the left side of the graph, just like the other right-skewed graphs. The numeric variable z represents the vertical height of the diamond (in mm) ranging from 2.40 mm to 6.24 mm with a mean of 3.54 mm and a median of 3.50 mm, this variable is mostly symmetric with a slight right skew. The categorical variable cut is the quality of the cut with levels of: Fair, Good, Ideal, Premium, and Very Good. The most common level is Ideal with a frequency of 412. The categorical variable color is the diamond color with D being the best and J being the worst. The most common color is G with a frequency of 204. Finally, the categorical variable clarity is the diamond clarity with the most common being SI1 with 266 diamonds and the least common being VVS1 and other with 56 and 42 diamonds respectively.

3) Determine if there is any correlation between these variables

```
continuous_vars <- diamonds_sample[, c("carat", "depth", "table", "price", "x", "y", "z")]
cor_mat <- cor(continuous_vars)
print(round(cor_mat, 2))
```

```
##       carat depth table price     x     y    z
## carat  1.00  0.00  0.17  0.93  0.97  0.97 0.97
## depth  0.00  1.00 -0.23 -0.01 -0.06 -0.06 0.08
## table  0.17 -0.23  1.00  0.12  0.18  0.17 0.14
## price  0.93 -0.01  0.12  1.00  0.90  0.90 0.90
## x      0.97 -0.06  0.18  0.90  1.00  1.00 0.99
## y      0.97 -0.06  0.17  0.90  1.00  1.00 0.99
## z      0.97  0.08  0.14  0.90  0.99  0.99 1.00
```

There is strong correlation between carat and price with a value of 0.93, indicating a strong positive relationship as heavier diamonds tend to be much more expensive.

There is a very strong correlation between carat and x,y,z because as carat increases, so do the physical dimensions.

Another strong correlation can be seen with x,y,z and price as larger diamonds in all measurements tend to cost more.

x,y,z have an almost perfect negative correlation as they all scale together, therefore only one may be needed in the model.

Depth has no linear relationship with price with a value of -0.01, therefore depth % has no impact on price.

Table and Price have a correlation value of 0.12 which is very weak.

Depth with any of the other variables has values very close to 0, resulting in little to no relationship with the other variables.

With that being said, x, y, and z are nearly perfectly correlated, so using all three in a regression model can cause multicollinearity. In that case, it may be best to consider only using one for modeling purposes. It is also possible that we should probably exclude depth and table from the model as they are weakly correlated to all other variables.

4) Run the Multiple Linear Regression Model

```
diamonds_sample$cut <- as.factor(diamonds_sample$cut)
diamonds_sample$color <- as.factor(diamonds_sample$color)
diamonds_sample$clarity <- as.factor(diamonds_sample$clarity)

model <- lm(price ~ ., data = diamonds_sample)
summary(model)
```

```
##
## Call:
## lm(formula = price ~ ., data = diamonds_sample)
##
## Residuals:
```

```
##      Min       1Q   Median       3Q      Max
## -10854.9   -567.7   -174.6    372.3   4880.0
##
## Coefficients:
##                Estimate Std. Error t value Pr(>|t|)
## (Intercept)  -2.324e+04  7.486e+03  -3.105 0.001960 **
## X             5.043e-03  2.600e-03   1.940 0.052715 .
## carat         9.791e+03  3.474e+02  28.188  < 2e-16 ***
## cutGood       6.635e+02  2.393e+02   2.773 0.005668 **
## cutIdeal      7.794e+02  2.315e+02   3.367 0.000789 ***
## cutPremium    8.418e+02  2.212e+02   3.805 0.000150 ***
## cutVery Good  5.614e+02  2.292e+02   2.449 0.014488 *
## colorE       -2.231e+02  1.296e+02  -1.721 0.085540 .
## colorF       -2.172e+02  1.300e+02  -1.670 0.095230 .
## colorG       -3.356e+02  1.268e+02  -2.647 0.008259 **
## colorH       -7.271e+02  1.364e+02  -5.332 1.21e-07 ***
## colorI       -1.235e+03  1.564e+02  -7.893 7.88e-15 ***
## colorJ       -2.289e+03  1.928e+02 -11.875  < 2e-16 ***
## clarityIF     5.598e+03  3.938e+02  14.215  < 2e-16 ***
## claritySI1    4.485e+03  3.333e+02  13.457  < 2e-16 ***
## claritySI2    3.728e+03  3.340e+02  11.162  < 2e-16 ***
## clarityVS1    5.296e+03  3.388e+02  15.632  < 2e-16 ***
## clarityVS2    5.136e+03  3.360e+02  15.287  < 2e-16 ***
## clarityVVS1   5.648e+03  3.629e+02  15.565  < 2e-16 ***
## clarityVVS2   5.659e+03  3.488e+02  16.225  < 2e-16 ***
## depth         3.018e+02  1.190e+02   2.536 0.011381 *
## table        -2.657e+01  1.972e+01  -1.347 0.178194
## x            -1.897e+03  1.018e+03  -1.863 0.062719 .
## y             4.816e+03  1.013e+03   4.753 2.31e-06 ***
## z            -5.562e+03  1.948e+03  -2.855 0.004399 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1129 on 975 degrees of freedom
## Multiple R-squared:  0.9229, Adjusted R-squared:  0.921
## F-statistic: 486.3 on 24 and 975 DF,  p-value: < 2.2e-16
```

Seeing as how we are not explaining hypothesis testing or anything else, we just have a normal summary output above with many signficant values, and no alarming features.

5) Comment on anything of interest that occurred in this part

The data was approximately how I expected with a few unexpected results. For example, I was not expecting depth to have such negative correlation with every other variable as based on the variable description and the way in which it is calculated, I assumed there would be some sort of correlation with at least a few variables, specifically x, y and z. Furthermore, the depth variable also has a significant p-value at $\alpha = 0.05$ which is even more peculiar due to the fact that based on the correlation matrix, it may not be useful in the model at all, although this could be because we haven't accounted for the Bonferroni Adjustment. So as to keep in line with the directions of #4, we will not go into the hypothesis testing side of it. Another result that surprised me involved the histograms and the amount of right-skewness present within our sample, however this is how the data was collected and therefore it is normal to see. Carat can be seen as the best single predictor of price with such a high value present within the correlation matrix, sitting at 0.93, which could be useful later in creating the best possible model.

Part 2: Simple Linear Regression

1) Start with one predictor and one response from variables, conduct SLR analysis

```r
slr <- lm(price ~ carat, data = diamonds_sample)
```

Starting with one predictor, ‘carat’, and the response variable ‘price’, and conducting a simple linear regression analysis on it, labeling the model as ‘slr’ for ease of writing and less clutter within the code chunks below.

2) Run the model and examine summary statistics, interpreting everything

```r
summary(slr)
```

```
##
## Call:
## lm(formula = price ~ carat, data = diamonds_sample)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -12670.3   -736.3     -9.5    505.8   6638.9
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept) -2103.23      88.66  -23.72   <2e-16 ***
## carat        7480.43      94.02   79.56   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1483 on 998 degrees of freedom
## Multiple R-squared:  0.8638, Adjusted R-squared:  0.8637
## F-statistic:  6330 on 1 and 998 DF,  p-value: < 2.2e-16
```

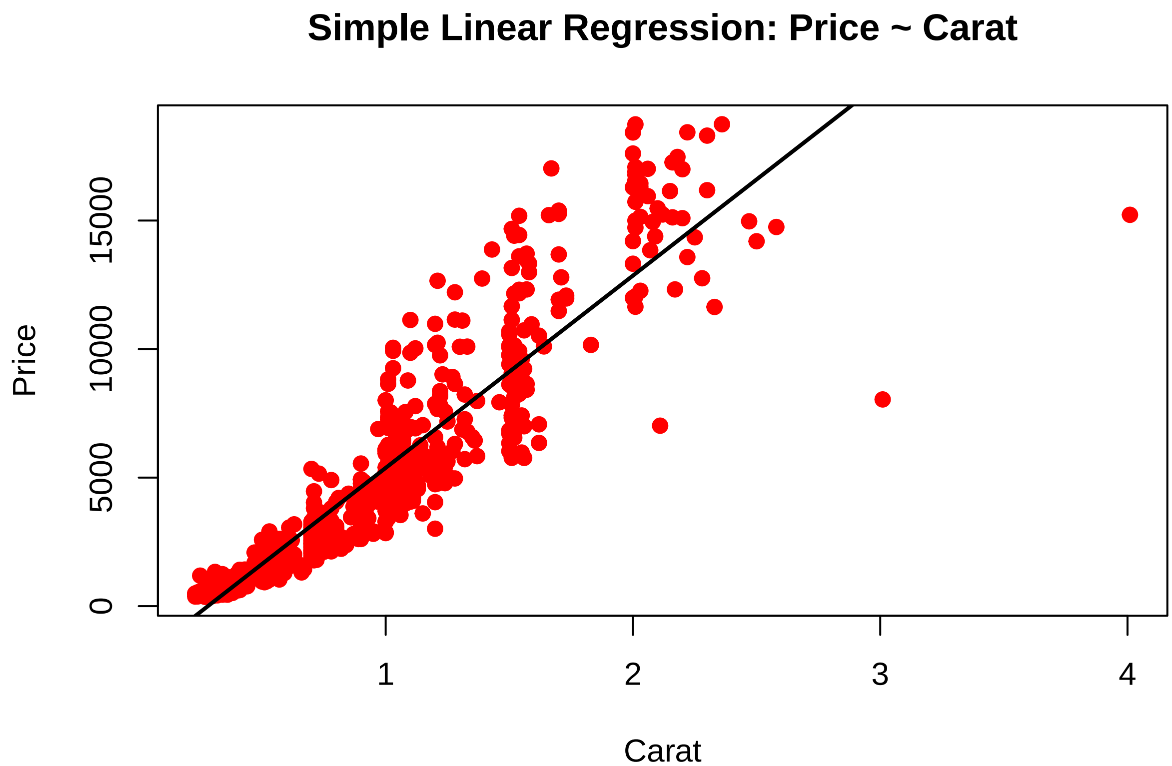
```r
confint(slr)
```

```
##                 2.5 %    97.5 %
## (Intercept) -2277.204 -1929.257
## carat        7295.935  7664.935
```

```r
predict(slr, newdata= data.frame(carat = 1.0), interval = "prediction")
```

```
##        fit      lwr      upr
## 1 5377.204 2465.425 8288.984
```

```r
plot(diamonds_sample$carat, diamonds_sample$price,
     main = "Simple Linear Regression: Price ~ Carat",
     xlab = "Carat", ylab = "Price", pch = 19, col = "red")
abline(slr, col="black", lwd=2)
```



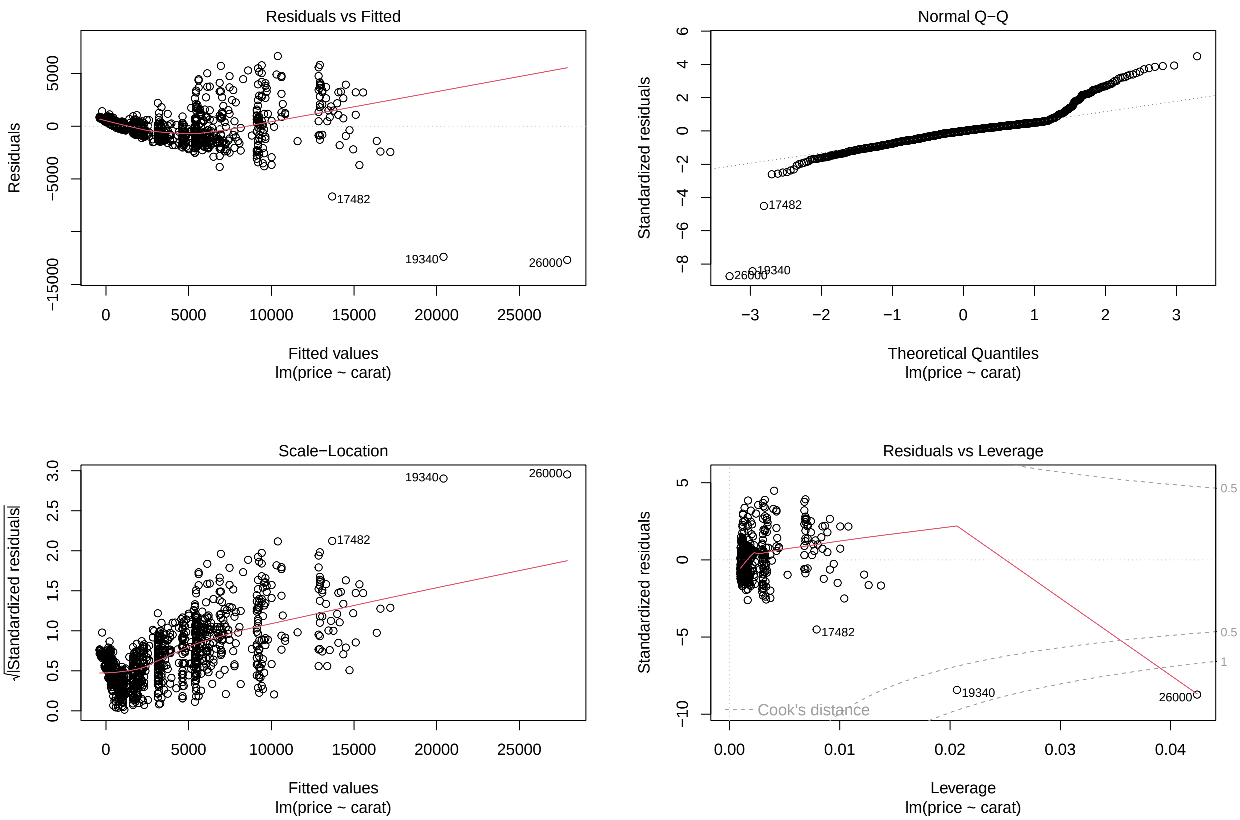
Running the model with one predictor 'carat' and the response variable 'price', then calling the summary function on it, we receive the first output above. We have an (Intercept) or baseline value, β_0, of -2103.23, with a p-value of <2e-16 which is highly significant. For the carat coefficietn we also have the same p-value, indicating that the variable is highly significant and therefore rejecting H_0 that carat has no effect on price and concluding that it has a statistically significant effect on price as $H_A : \beta_1 \neq 0$. The estimate for carat is 7480.43 which means that for each additional carat (one unit increase), the price increases by approximately $7,480.43 on average. Our R^2 value is 0.8638 meaning that approximately 86.38% of the variance in price is explained by carat. The adjusted R^2 value is 0.8637, very close to R^2 due to the model only having one predictor. Adjusted R^2 is used when you have multiple predictors as it penalizes for adding unecessary variables, as discussed in class. The conclusion based off the summary output is that this model is a very strong model, especially for simple linear regression, which is further backed up by the p-value for the overall model with a value of $<$ 2.2e-16.

For the intercept, we are 95% confident that the true y-intercept, or the price when carat=0) lies between -2277.20 and -1929.26, however this is not meaningful due to the fact that diamonds can't have zero carat. For the slope, or carat, we are 95% confident that the true increase in price per additional carat lies between $7,295.94 and $7,664.94. Since the range does not include zero, it confirms that carat is a statistically significant predictor. The model predicts that a diamond of carat size 1 will cost about $5,377.20. However, due to individual variability, we expect the actual price to fall between $2,465.43 and $8,288.98 about 95% of the time.

The plot shows a positive relationship, with the regression line sloping upward, between carat and price. So, as carat increases, the price also tends to increase. The points are generally clustered around the regression line for lower carat values, mostly below 2. For larger carats, greater than 2, there is more variability in prices, suggesting that price becomes less predictable as carat increases, possibly due to an influence from other variables in the full model. Carat is a strong predictor of price, but it does not explain all the variation, therefore there may be non-linearity or non-constant variance as carat increases, thus including more predictors could help to improve the model.

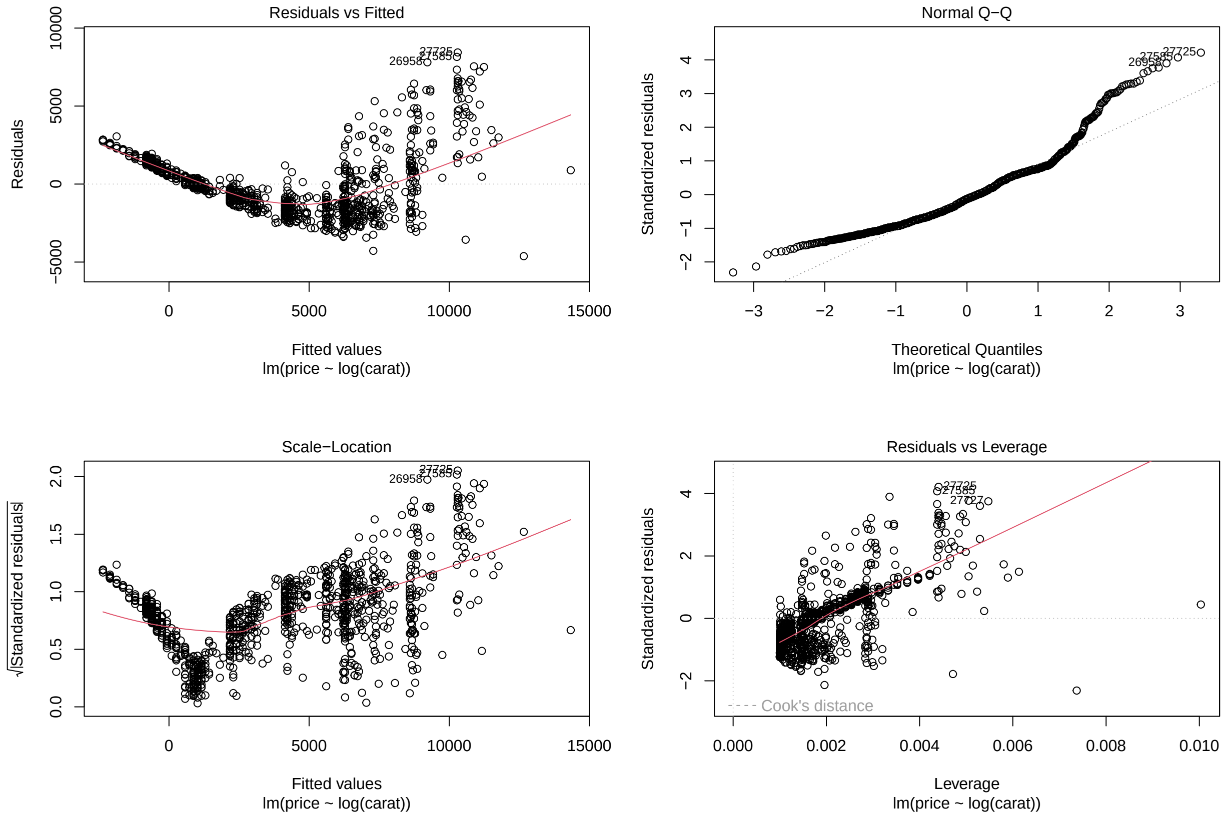
3) Test the Assumptions and Apply Necessary Transformations to the response variable y or the predictor

```r
plot(slr)
```



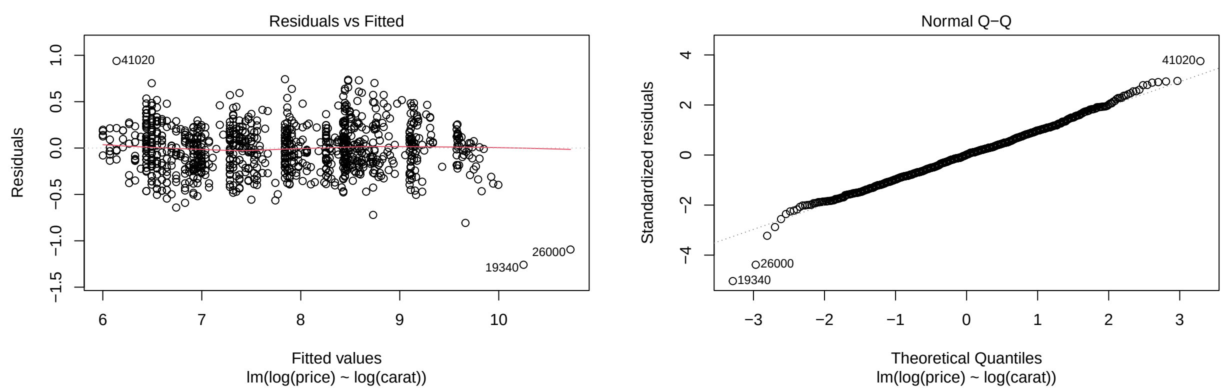
Here we have plots that help us to determine if the assumptions for a linear model are upheld or not. Here we see a departure from linearity within the first plot, as the red line or the LOESS smoother (LOcal regrESSion, also called lowess) line helps us to visualize the trend or pattern in residuals. Specifically, the linearity assumption holds if the red line is flat and close to 0 across the range of fitted values. If the red line curves, dips or rises, the linearity assumption is violated. Homoscedasticity is violated if the red line stays flat but residual spread increases/decreases around it. Here we see that the linearity assumption is clearly violated, therefore we have to transform the model. We start by first transforming the independent variable.

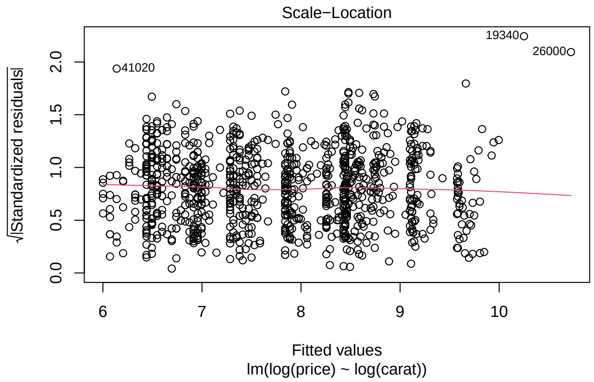
```r
#transforming the independent variable first
model_transpred <- lm(price ~ log(carat), diamonds_sample)
plot(model_transpred)
```

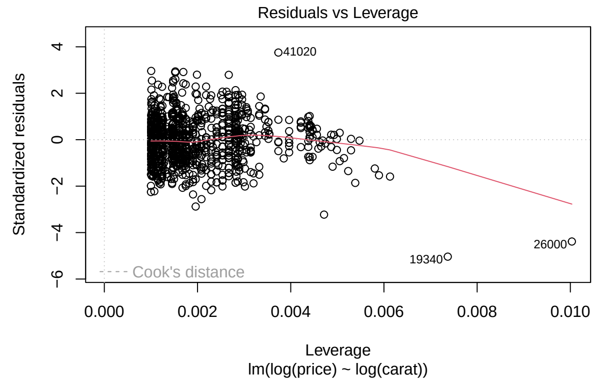


Here we see that the red line dips, and therefore the linearity assumption is still violated, so we move onto transforming both variables.

```r
#transforming both
model_transresp <- lm(log(price) ~ log(carat), diamonds_sample)
plot(model_transresp)
#NOTE ANY CHANGES IN THE TRANSFORMED MODELS
```







A log-log transformation was applied to correct for non-linearity and heteroscedasticity in the original model. After transforming both variables, we see that the line has flattened out significantly from the previous two plots, thus the log transformation of both variables has successfully addressed the key model assumptions of Linearity, Homoscedasticity, and Normality of Residuals with a few points having influence but not being problematic.

4) Call the summary function on the transformed variables, observe, and note any changes

```r
summary(slr)
```

```
##
## Call:
## lm(formula = price ~ carat, data = diamonds_sample)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -12670.3   -736.3     -9.5    505.8   6638.9
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept) -2103.23      88.66  -23.72   <2e-16 ***
## carat        7480.43      94.02   79.56   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1483 on 998 degrees of freedom
## Multiple R-squared:  0.8638, Adjusted R-squared:  0.8637
## F-statistic:  6330 on 1 and 998 DF,  p-value: < 2.2e-16
```

```r
summary(model_transpred)
```

```
##
## Call:
## lm(formula = price ~ log(carat), data = diamonds_sample)
##
## Residuals:
```

```
##     Min      1Q  Median      3Q     Max
## -4621.2 -1465.5  -264.1  1156.7  8437.8
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept)  6226.43      76.51   81.38   <2e-16 ***
## log(carat)   5839.52     106.53   54.82   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 2007 on 998 degrees of freedom
## Multiple R-squared:  0.7507, Adjusted R-squared:  0.7504
## F-statistic:  3005 on 1 and 998 DF,  p-value: < 2.2e-16
```

```r
summary(model_transresp)
```

```
##
## Call:
## lm(formula = log(price) ~ log(carat), data = diamonds_sample)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1.25838 -0.17135  0.00395  0.16113  0.93933
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept) 8.429719   0.009563   881.5   <2e-16 ***
## log(carat)  1.652396   0.013314   124.1   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.2508 on 998 degrees of freedom
## Multiple R-squared:  0.9391, Adjusted R-squared:  0.9391
## F-statistic: 1.54e+04 on 1 and 998 DF,  p-value: < 2.2e-16
```

In all summary outputs, from the original to the two transformed models, we have a constant p-value across all models for each coefficient and the overall model at <2e-16 and < 2.2e-16 respectively. The drastic changes that can be seen throughout each model are the differences within the coefficients of each model, when we only transform the predictor, carat, the estimate for the intercept changes from -2103.23 to 6226.43 along with the standard error changing from 88.66 to 76.51 and the t-value changing from -23.72 to 81.38. Furthermore, the estimate for carat decreases to 5839.52 with an increased standard error of 106.53 and a decreased t-value of 54.82. The R^2 and adjusted R^2 values also decrease to 0.7507 and 0.7504 respectively, indicating that this is a worse model than the original.

When we perform a log-log transformation on the model, transforming both the response and the predictor, we get drastic differences in coefficient values once again, but this time our R^2 and adjusted R^2 increase to 0.9391 for both, indicating that this model has improved from the original, and should be the model to be used for further analysis.

5) Improving the Model

After adding the other variables to the model and assessing if the model improves, it was found that cut, color, and clarity all provide a significant increase in the adjusted R^2 of our model and should therefore be included in our model. Depth, table, x, y, and z all provide either insignificant or no difference in the adjusted R^2 and should be excluded from the model. For example, z only provided an increase of 0.0001 which is considered negligible and not worth including in the model, despite an increase in adjusted R^2.

The variable cut provided a +0.0022 increase in adjusted R^2

The variable color provided a +0.0107 increase in adjusted R^2 after cut was added

The variable clarity provided a +0.0278 increase in adjusted R^2 after cut and color were added to the model.

6) Comment on anything of interest that occurred

It was interesting to see the individual effects on adjusted R^2 that each variable had on the model, and deciding whether to exclude a variable or not was an interesting process. Just because it provides an increase in adjusted R^2 doesn't mean that it is a significant enough increase to be included in our final model. Therefore x,y, and z were all excluded from the model as they didn't provide significant enough differences in adjusted R^2, or no difference at all, and therefore are not worth including. This process helped to pinpoint exactly which variables are the most important for our model and figuring out the price of a diamond. In this case, it was cut, color, and clarity. It was also interesting to analyze and compare the graphs of the transformed models and the original model, checking each assumption in order for a model to be accurate and as precise as possible, ultimately ending up transforming both the predictor and response variable, giving us the best outcome possible and passing the checks on the assumptions.

Part 3: Continuation of Part 2…

1) AIC - Backward Elimination

```r
# AIC - Backward Elimination
full_model <- lm(log(price) ~ log(carat) + cut + color + clarity, diamonds_sample)
best_model_aic <- step(full_model, direction = "backward")
```

```
## Start:  AIC=-3851.62
## log(price) ~ log(carat) + cut + color + clarity
##
##              Df Sum of Sq    RSS     AIC
## <none>                     20.45 -3851.6
## - cut         4      1.43  21.89 -3791.9
## - color       6     14.70  35.15 -3322.0
## - clarity     7     28.49  48.94 -2993.1
## - log(carat)  1    858.00 878.45   -93.6
```

```r
test <- lm(log(price) ~ log(carat) + cut + color, diamonds_sample)
step(test, direction="backward")
```

```
## Start:  AIC=-2993.06
## log(price) ~ log(carat) + cut + color
##
##              Df Sum of Sq    RSS      AIC
## <none>                     48.94 -2993.06
## - cut         4      2.78  51.72 -2945.83
## - color       6     11.34  60.29 -2796.60
## - log(carat)  1    908.73 957.67   -21.25
```

```
##
## Call:
## lm(formula = log(price) ~ log(carat) + cut + color, data = diamonds_sample)
##
## Coefficients:
##  (Intercept)    log(carat)       cutGood      cutIdeal    cutPremium
##      8.38525       1.71839       0.13260       0.23328       0.17718
## cutVery Good        colorE        colorF        colorG        colorH
##      0.14053      -0.04109      -0.05192      -0.08279      -0.18065
##       colorI        colorJ
##     -0.28544      -0.44548
```

When conducting backward elimination using AIC to find the best possible model for the dataset, we want the smallest AIC value we can get. So, after running backward elimination using AIC on the full_model that we obtained in the previous section, we get an AIC value of -3851.62, which is incredibly low and exactly what we require for the best possible model. To further solidify this, another test was run in order to confirm that this is the best model by removing one of the predictors. When this was performed, we received an AIC value significantly higher than before at -2993.06, further solidifying our claim for the best model. Additionally, several other "sanity checks" were run in order to back up our full model. For example, a test was run on the model without the transformations which gave us a significantly higher AIC value, and a test was run on our full model with the transformations using stepwise regression instead of AIC, which gave us the same answer as the AIC backward elimination. These sanity checks were ran in the background and not included in the final code here.

2) Detect Multicollinearity among the variables using VIF

```r
library(car)
```

```
## Loading required package: carData
```

```r
vif(best_model_aic)
```

```
##                GVIF Df GVIF^(1/(2*Df))
## log(carat) 1.438322  1        1.199300
## cut        1.107396  4        1.012833
## color      1.240095  6        1.018094
## clarity    1.417134  7        1.025215
```

```r
#VIF > 5 or 10 indicates multicollinearity concern, comment on any variables with high VIFs and whether
```

No variables in our model exhibit any kind of multicollinearity concern, with all VIF values, specifically GVIF^(1/(2*Df)), being less than 5, further proving that this model is in fact the best possible model.

3) CIs from Final Model

```r
new_data <- data.frame(
  carat = 1,
  cut = "Ideal",
```

```r
  color = "G",
  clarity = "VS2"
)
new_data$cut <- factor(new_data$cut, levels = levels(diamonds_sample$cut))
new_data$color <- factor(new_data$color, levels = levels(diamonds_sample$color))
new_data$clarity <- factor(new_data$clarity, levels = levels(diamonds_sample$clarity)) # making sure an
predict(full_model, newdata = new_data, interval = "confidence") # confidence interval for the mean log
```

```
##        fit    lwr     upr
## 1 8.594755 8.5643 8.62521
```

```r
predict(full_model, newdata = new_data, interval = "prediction") # prediction interval for a new indivi
```

```
##        fit      lwr      upr
## 1 8.594755 8.309769 8.879741
```

Since the model predicts log(price), we can back-transform using exp() to give us the desired values for a confidence and prediction interval:

```r
exp(predict(full_model, newdata = new_data, interval = "confidence"))
```

```
##        fit      lwr     upr
## 1 5403.245 5241.171 5570.33
```

```r
exp(predict(full_model, newdata = new_data, interval = "prediction"))
```

```
##        fit      lwr      upr
## 1 5403.245 4063.375 7184.927
```

Our model fits a diamond with the characteristics of: 1 carat, an 'Ideal' cut, a color of 'G', and a clarity of 'VS2' with an expected price of $5,403.24. We are 95% confident that the mean price of the diamonds with these characteristics lies between $5,241.17 and $5,570.33. Additionally, we predict that a single new diamond with the same characteristics will cost between $4,063.38 and $7,184.93, 95% of the time based on our prediction interval.

Another set of CIs for a mean predicted value and the PIs for a future predicted value with another combination of variables:

```r
new_data2 <- data.frame(
  carat = 2,
  cut = "Premium",
  color = "E",
  clarity = "SI1"
)

new_data2$cut <- factor(new_data2$cut, levels = levels(diamonds_sample$cut))
new_data2$color <- factor(new_data2$color, levels = levels(diamonds_sample$color))
new_data2$clarity <- factor(new_data2$clarity, levels = levels(diamonds_sample$clarity)) # making sure 
predict(full_model, newdata = new_data2, interval = "confidence") # confidence interval for the mean lo
```

```
##        fit      lwr      upr
## 1 9.827667 9.790462 9.864872
```

```r
predict(full_model, newdata = new_data2, interval = "prediction") # prediction interval for a new indiv
```

```
##        fit      lwr      upr
## 1 9.827667 9.541881 10.11345
```

```r
exp(predict(full_model, newdata = new_data2, interval = "confidence"))
```

```
##        fit      lwr     upr
## 1 18539.65 17862.56 19242.4
```

```r
exp(predict(full_model, newdata = new_data2, interval = "prediction"))
```

```
##        fit      lwr     upr
## 1 18539.65 13931.13 24672.7
```

Our model fits a diamond with the characteristics of: 2 carat, a 'Premium' cut, a color of 'E', and a clarity of 'SI1' with an expected price of $18,539.65. We are 95% confident that the mean price of the diamonds with these characteristics lies between $17,862.56 and $19,242.40. Additionially, we predict that a single new diamond with the same characteristics will cost between $13,931.13 and $24,672.70, 95% of the time based on our prediction interval.

4) Summarize your report (for the final deliverable)

The purpose of this analysis was to build the best possible predictive model for diamond prices using a real-world dataset, based on a sample of 1,000 observations which was the minimum for this analysis. The goal was to accurately predict diamond prices based on characteristics drawn from previously collected data. The final model we obtained can be used to estimate a reasonable price range for any diamond given its attributes. Several statistical techniques were employed, including model diagnostics, visualizations (histograms, bar plots, and residual plots), as well as variable selection tools such as AIC and VIF. Confidence and prediction intervals were also used to determine the reliability of the model and provide estimates of the price of the diamond. Initial models were inaccurate, calling for a log-log transformation that significantly improved the fit of the model. The final model included carat, cut, color, and clarity as the strongest predictors of price, while all other variables were removed due to insignificance. These four predictors had the strongest impact and led to a strong model capable of generating accurate and meaningful predictions. A limitation in this analysis is the use of only 1,000 observations from a much larger dataset. However, the model offers valuable insight into the factors that produce diamond prices, and could be further improved and refined by exploring additional predictors or nonlinear relationships.